



Detección de NLoS basada en IA para SoCs de UWB

Autor:

Esp. Ing. Lautaro Vera

Director:

A designar (FIUBA)

*Esta planificación fue realizada en el curso de Gestión de proyectos
entre el 25 de agosto de 2026 y el 27 de octubre de 2026.*

Índice

1. Descripción técnica-conceptual del proyecto a realizar	5
2. Identificación y análisis de los interesados	6
3. Propósito del proyecto	7
4. Alcance del proyecto	7
5. Supuestos del proyecto.	9
6. Product Backlog	9
7. Criterios de aceptación de historias de usuario	10
8. Fases de CRISP-DM	11
9. Desglose del trabajo en tareas	11
10. Planificación de Sprints	12
11. Diagrama de Gantt (sprints)	14
12. Gobernanza de datos	15
12.1 Cumplimiento normativo	15
12.2 Ética en el uso de inteligencia artificial	15
13. Gestión de riesgos	16
14. Sprint Review	17
15. Sprint Retrospective	18

Registros de cambios

Revisión	Detalles de los cambios realizados	Fecha
0	Creación del documento	25 de agosto de 2026
1	Se completa hasta la sección 5 inclusive	8 de septiembre de 2026

Acta de constitución del proyecto

Buenos Aires, 25 de agosto de 2026

Por medio de la presente se acuerda con el Esp. Ing. Lautaro Vera que su Trabajo Final de la Maestría en Inteligencia Artificial Embebida se titulará “Detección de NLoS basada en IA para SoCs de UWB” y consistirá en el desarrollo de un modelo de aprendizaje profundo de bajo consumo de memoria, capaz de clasificar la respuesta al impulso del canal (CIR, del inglés *channel impulse response*) de un enlace de banda ultraancho (UWB, del inglés *ultra-wideband*) según exista o no línea de visión entre los dispositivos (LoS y NLoS, del inglés *line-of-sight* y *non-line-of-sight*), junto con el prototipo de firmware que ejecuta su inferencia sobre un SoC (del inglés *system on chip*) basado en ARM Cortex-M33. El trabajo tendrá un presupuesto preliminar estimado de 600 horas y un costo estimado de **USD 15.000**, con fecha de inicio el 25 de agosto de 2026 y fecha de presentación pública el 15 de noviembre de 2027.

Se adjunta a esta acta la planificación inicial.

Dr. Ing. Ariel Lutenberg
Director posgrado FIUBA

Carlos Paternain
MobileKnowledge

A designar
Director del Trabajo Final

1. Descripción técnica-conceptual del proyecto a realizar

Los transceptores UWB permiten medir distancias con una precisión del orden del centímetro y sostienen buena parte de los sistemas de localización en tiempo real (RTLS, del inglés *real-time locating system*) que se despliegan hoy en logística, control de acceso y llave digital vehicular. Esa precisión se degrada de manera significativa cuando el trayecto directo entre los dos dispositivos se encuentra obstruido, condición conocida como NLoS. En esa situación el receptor identifica como primer camino una réplica reflejada de la señal y la distancia estimada resulta mayor que la real, con errores que pueden alcanzar varios decímetros. La detección confiable de la condición NLoS es, por lo tanto, la principal palanca disponible para mejorar la exactitud del *ranging*.

En los productos actuales esa detección se resuelve mediante algoritmos de procesamiento digital de señales integrados en el propio chip, que aplican reglas deterministas sobre un conjunto reducido de métricas de canal, como la amplitud del primer camino o la potencia total recibida. El enfoque tiene la ventaja del determinismo y del bajo consumo, pero descarta la mayor parte de la información contenida en la respuesta al impulso del canal. En paralelo, la literatura reciente muestra que los modelos de aprendizaje profundo aplicados sobre la CIR completa alcanzan tasas de acierto muy superiores. Esos trabajos, entre ellos la arquitectura ADCNN-BiLSTM-Transformer publicada en *IEEE Journals*, se ejecutan fuera del chip, sobre un procesador de propósito general, porque su tamaño excede ampliamente la memoria disponible en un transceptor UWB.

El proyecto se ubica en ese espacio intermedio. Se propone desarrollar una red neuronal convolucional (CNN, del inglés *convolutional neural network*) unidimensional de bajo perfil en memoria, cuantizado a enteros de 8 bits, que clasifique la CIR como LoS o NLoS con una exactitud comparable a la de los modelos publicados, pero con un tamaño compatible con la memoria de un dispositivo embebido. El entregable es un producto mínimo viable que demuestra la factibilidad de trasladar la inferencia al borde, como paso previo a su integración en un transceptor comercial de la familia NXP SR250 o NCJ29D6.

El contexto es empresarial. El trabajo se realiza en MobileKnowledge, compañía especializada en tecnologías de conectividad de corto alcance y socia tecnológica de NXP Semiconductors. La empresa aporta el conocimiento de dominio en UWB y el acceso al hardware de evaluación. El proyecto no cuenta con financiamiento público ni privado específico. La documentación técnica, el código de ejemplo y las especificaciones de los transceptores de NXP que se emplean son de acceso público, de modo que ninguna restricción de confidencialidad recae sobre el material técnico del proyecto. La propiedad intelectual del modelo que se desarrolle corresponde a MobileKnowledge, por lo que la difusión de los resultados se acuerda con la empresa.

La propuesta de valor se apoya en tres puntos. Primero, la solución opera sobre datos que el transceptor ya calcula, sin necesidad de sensores adicionales ni de cambios en el protocolo de *ranging*. Segundo, al ejecutarse en el dispositivo elimina la latencia y el consumo asociados a enviar la CIR completa a un procesador externo, que por su longitud representa un volumen de datos considerable para un enlace embebido. Tercero, el modelo se diseña desde el inicio con un presupuesto de memoria explícito, criterio ausente en los trabajos académicos de referencia, que priorizan la exactitud sin restricciones de plataforma.

En la figura 1 se presenta el diagrama en bloques del sistema. Se observan dos cadenas de procesamiento. La cadena superior corresponde al entorno de desarrollo, donde el conjunto de datos público se prepara, se emplea para entrenar la red convolucional, y el modelo resultante se cuantiza y se convierte en un arreglo de C que se enlaza con el firmware. La cadena inferior corresponde al sistema embebido: el transceptor UWB entrega la CIR de cada intercambio de

ranging, el firmware la acondiciona, el motor de inferencia produce la decisión entre LoS y NLoS, y esa decisión se utiliza para ponderar o descartar la medición de distancia antes de entregarla a la aplicación de localización.

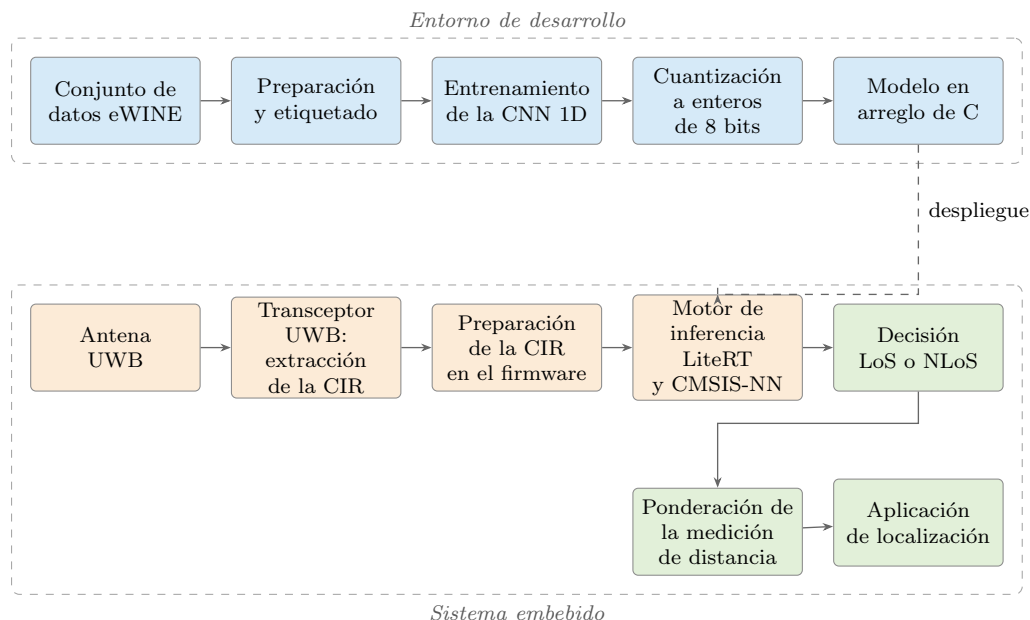


Figura 1. Diagrama en bloques del sistema.

Los bloques de la cadena de desarrollo se ejecutan una sola vez, durante el proyecto, sobre una computadora personal. Los bloques de la cadena embebida se ejecutan en cada intercambio de *ranging* y son los que imponen las restricciones de memoria y de tiempo de respuesta que condicionan el diseño del modelo. El bloque de preparación de la CIR resuelve la alineación de la CIR respecto del primer camino detectado y su normalización, y es la pieza que garantiza que el modelo reciba una entrada equivalente a la que se usó durante el entrenamiento.

En términos de ciclo de vida, la solución se encuentra en etapa de prueba de concepto. El proyecto la lleva hasta un prototipo validado sobre hardware de evaluación, con métricas de exactitud, memoria y latencia medidas. La integración en un producto comercial queda fuera del alcance y depende de decisiones posteriores de la empresa.

2. Identificación y análisis de los interesados

En la tabla 1 se identifican los interesados del proyecto y el rol que ocupa cada uno.

Cuadro 1. Identificación de los interesados.

Rol	Nombre y Apellido	Organización	Puesto
Cliente	Carlos Paternain	MobileKnowledge	Responsable técnico
Responsable	Esp. Ing. Lautaro Vera	FIUBA	Alumno
Orientador	A designar	FIUBA	Director del Trabajo Final
Colaboradores	Equipo de ingeniería UWB	MobileKnowledge	Ingenieros de aplicación
Usuario final	Integradores de sistemas RTLS y desarrolladores de firmware UWB	Empresas del sector	-

A continuación se describen las características de cada interesado que resultan relevantes para la planificación.

- Cliente: Carlos Paternain aprueba los entregables y participa en la definición de los criterios de aceptación. Conoce en profundidad la tecnología UWB y su mercado, por lo que su revisión es determinante para validar que la planificación sea realista.
- Responsable: el alumno ejecuta la totalidad de las tareas técnicas y de documentación. Cursa el posgrado en paralelo con su actividad profesional, condición que fija la dedicación semanal disponible y que se refleja en el cronograma.
- Orientador: el director del Trabajo Final orienta las decisiones metodológicas, en particular el diseño de la validación y la interpretación de las métricas. Se prevén encuentros de seguimiento al cierre de cada bloque de sprints.
- Colaboradores: el equipo de ingeniería UWB de MobileKnowledge facilita el acceso al hardware de evaluación y responde consultas sobre el formato de la CIR. Su participación es puntual y no se le asignan tareas del cronograma.
- Usuario final: los integradores de sistemas RTLS y los desarrolladores de firmware UWB son quienes adoptarían la solución. Valorán la exactitud del *ranging*, el consumo de memoria y la ausencia de cambios en el protocolo existente.

3. Propósito del proyecto

El propósito de este proyecto es demostrar que la detección de la condición NLoS en enlaces UWB puede resolverse con un modelo de aprendizaje profundo lo suficientemente pequeño como para ejecutarse dentro de un dispositivo embebido, y no solo en un procesador externo, de manera de mejorar la exactitud de la medición de distancia sin agregar hardware, sin modificar el protocolo de *ranging* y sin transferir la respuesta al impulso del canal fuera del dispositivo.

4. Alcance del proyecto

El proyecto incluye:

- La construcción de un flujo reproducible de ingesta del conjunto de datos público eWINE, con particiones versionadas para entrenamiento, validación y prueba.
- El análisis exploratorio de las respuestas al impulso del canal, discriminado por clase y por entorno de medición, y la documentación de los sesgos detectados.
- El diseño e implementación del preprocesamiento de la CIR, que comprende:
 - La alineación respecto del primer camino detectado.
 - La normalización de amplitud.
 - La selección del ancho de ventana que minimiza el uso de memoria sin degradar la exactitud.
- El entrenamiento de una red convolucional unidimensional de referencia y de dos modelos de comparación clásicos sobre las métricas de canal.
- La exploración de arquitecturas sujeta a un presupuesto de memoria explícito, con poda estructurada del modelo seleccionado.
- La cuantización del modelo a enteros de 8 bits y la verificación de compatibilidad con el conjunto de operadores de LiteRT for Microcontrollers, denominado TensorFlow Lite for Microcontrollers hasta su cambio de marca.
- El desarrollo del firmware de prueba que ejecuta la inferencia sobre un SoC basado en ARM Cortex-M33, con aceleración mediante CMSIS-NN (Common Microcontroller Software Interface Standard, Neural Networks), la biblioteca de núcleos de redes neuronales que ARM optimiza para esos procesadores.
- La medición sobre el hardware de evaluación de la memoria de programa ocupada, la memoria de trabajo del intérprete, la latencia de inferencia y la exactitud alcanzada.
- La comparación de los resultados del modelo contra el criterio determinista de referencia disponible en el transceptor.
- La elaboración del informe de avance y de la memoria técnica.

El presente proyecto no incluye:

- La realización de campañas de medición propias para generar un conjunto de datos nuevo.
- La integración del modelo dentro del firmware de producción de los transceptores SR250 o NCJ29D6.
- El desarrollo de una aplicación de localización en tiempo real completa.
- La industrialización, la fabricación de hardware a medida y cualquier actividad de puesta en el mercado.
- La detección o corrección del error de *ranging* en magnitud, que constituye un problema de regresión distinto del problema de clasificación abordado.

5. Supuestos del proyecto

Para el desarrollo del presente proyecto se supone que:

- El alumno dispone de al menos 16 horas semanales durante los períodos activos del cronograma, y de las condiciones de salud necesarias para sostener esa dedicación.
- MobileKnowledge mantiene el interés en el proyecto y la asignación del cliente durante todo el período, sin que un cambio de prioridades comerciales interrumpa las instancias de revisión.
- La empresa provee la placa de evaluación con un SoC basado en ARM Cortex-M33 y su cadena de compilación durante toda la etapa de despliegue.
- El conjunto de datos eWINE permanece disponible en su repositorio público y sus condiciones de uso no se modifican de manera restrictiva.
- La respuesta al impulso del canal contiene información suficiente para discriminar las condiciones LoS y NLoS con la exactitud objetivo, tal como reportan los trabajos publicados sobre el mismo conjunto de datos.
- Las herramientas de código abierto empleadas, entre ellas TensorFlow, LiteRT for Microcontrollers y CMSIS-NN, conservan sus licencias permisivas actuales.
- El equipamiento de cómputo disponible resulta suficiente para entrenar los modelos previstos, cuyo tamaño es reducido, sin necesidad de contratar infraestructura en la nube.

6. Product Backlog

El Product Backlog debe organizarse en cuatro *épicas* fundamentales del proyecto. Cada *épica* debe contener al menos dos historias de usuario que describan funcionalidades clave.

El Product Backlog debe permitir interpretar cómo será el proyecto y su funcionalidad. Se deben indicar claramente las prioridades entre las historias de usuario y si hay alguna opcional.

Las historias de usuario deben ser breves, claras y medibles, expresando el rol, la necesidad y el propósito de cada funcionalidad. También deben tener una prioridad definida para facilitar la planificación de los sprints.

Cada historia de usuario debe incluir una ponderación en *Story Points*, un número entero que representa el tamaño relativo de la historia. El criterio para calcular los Story Points debe indicarse explícitamente.

Las historias deben seguir el formato: “*Como [rol], quiero [tal cosa] para [tal otra cosa]*”.

Las épicas deben estructurarse de la siguiente forma:

- **Épica 1**
 - HU1
 - HU2

- **Épica 2**

- HU3
- HU4

- **Épica 3**

- HU5
- HU6

- **Épica 4**

- HU7
- HU8

Reglas para definir historias de usuario:

- Ser concisas y claras.
- Expresarlas en términos cuantificables y medibles.
- No dejar margen para interpretaciones ambiguas.
- Indicar claramente su prioridad y si son opcionales.
- Considerar regulaciones y normas vigentes.

7. Criterios de aceptación de historias de usuario

Los criterios de aceptación deben establecerse para cada historia de usuario, asegurando que se cumplan las condiciones necesarias para que la funcionalidad sea validada correctamente.

Cada historia debe tener criterios medibles, específicos y verificables. Deben permitir validar que se cumple con las necesidades del usuario.

Se estructuran de forma análoga a las épicas del backlog:

- **Épica 1**

- Criterios de aceptación HU1
- Criterios de aceptación HU2

- **Épica 2**

- Criterios de aceptación HU3
- Criterios de aceptación HU4

- **Épica 3**

- Criterios de aceptación HU5
- Criterios de aceptación HU6

■ Épica 4

- Criterios de aceptación HU7
- Criterios de aceptación HU8

Reglas para definir criterios de aceptación:

- Medibles y verificables.
- Especificar cuándo una historia se considera completada.
- Incluir condiciones específicas.
- No ambiguos.
- Probables de testear funcional o técnicamente.
- Mínimo 3 criterios por HU.

8. Fases de CRISP-DM

1. **Comprensión del negocio:** objetivo, valor agregado de IA, métricas de éxito.
2. **Comprensión de los datos:** tipo, origen, cantidad, calidad.
3. **Preparación de los datos:** características clave, transformaciones necesarias.
4. **Modelado:** tipo de problema, algoritmos posibles.
5. **Evaluación del modelo:** métricas de rendimiento.
6. **Despliegue del modelo (opcional):** tipo de despliegue y herramientas.

9. Desglose del trabajo en tareas

A partir de cada Historia de Usuario (HU) definida en la sección 6, descomponer el trabajo en tareas técnicas concretas, medibles y acotadas en el tiempo.

- Seleccionar entre 2 y 3 tareas por cada historia de usuario.
- Cada tarea debe estar claramente formulada, ser técnica, accionable y con una estimación horaria entre 2 y 8 horas.
- Evitar tareas genéricas (como "desarrollar funcionalidad") o demasiado amplias.
- Si una tarea supera las 8 horas, debe dividirse en subtareas.
- Indicar la prioridad relativa de cada tarea (Alta, Media o Baja).

Historia de usuario	Tarea técnica	Estimación	Prioridad
HU1	Tarea 1 HU1	6 h	Alta
HU1	Tarea 2 HU1	8 h	Alta
HU2	Tarea 1 HU2	5 h	Media
HU2	Tarea 2 HU2	6 h	Alta
...	...	...	...

Criterios para estimar tiempos:

- Considerar la complejidad técnica, el nivel de incertidumbre y la experiencia previa.
- Evitar subestimar el esfuerzo: estimar el tiempo realista que llevaría implementar, testear y documentar cada tarea.
- Basar la estimación en la experiencia propia o en referencias de tareas similares.

Sobre la prioridad:

- Asignar una prioridad relativa (Alta, Media o Baja) según la relevancia funcional de la tarea y su impacto en los entregables.
- Priorizar tareas que estén vinculadas a criterios de aceptación de las HU o que sean necesarias para desbloquear otras.
- Incluir tareas opcionales solo si están bien justificadas.

Recomendaciones generales:

- -Enfocarse en tareas que surgen directamente de las HU planteadas.
- No es necesario cubrir las 600 horas del proyecto en esta sección: el foco está en el desglose de funcionalidades clave.
- Este trabajo será la base para organizar algunos de los sprints y elaborar el cronograma del proyecto, por lo que debe ser claro y realista.

10. Planificación de Sprints

Organizar las tareas técnicas del proyecto en sprints de trabajo que permitan distribuir de forma equilibrada la carga horaria total, estimada en 600 horas.

Consigna:

- Completar una tabla que relacione sprints con HU y tareas técnicas correspondientes.
- Incluir estimación en horas para cada tarea.
- Indicar responsable y porcentaje de avance estimado o completado.

- Contemplar también tareas de planificación, documentación, redacción de memoria y preparación de defensa.

Conceptos clave:

- Una épica es una unidad funcional amplia; una historia de usuario es una funcionalidad concreta; un sprint es una unidad de tiempo donde se ejecutan tareas.
- Las tareas son el nivel más desagregado: permiten estimar tiempos, asignar responsables y monitorear progreso.

Duración sugerida:

- Para un proyecto de 600 h, se recomienda planificar entre 10 y 12 sprints de aproximadamente 2 semanas cada uno.
- Asignar entre 45 y 50 horas efectivas por sprint a tareas técnicas.
- Reservar 100 a 120 h para actividades no técnicas (planificación, escritura, reuniones, defensa).

Importante:

- En proyectos individuales, el responsable suele ser el propio autor.
- Aun así, desagregar tareas facilita el seguimiento y mejora continua.

Conversión opcional de Story Points a horas:

- $1 \text{ SP} \approx 2 \text{ h}$ como referencia flexible.
- Tener en cuenta aproximaciones tipo Fibonacci.

Recomendaciones:

- Verificar que la carga horaria por sprint sea equilibrada.
- Usar sprints de 1 a 3 semanas, acordes al cronograma general.
- Actualizar el % completado durante el seguimiento del proyecto.
- Considerar un sprint final exclusivo para pruebas, revisión y ajustes antes de la defensa.

Cuadro 2. Formato sugerido

Sprint	HU o fase	Tarea	Horas / SP	Responsable	% Completado
Sprint 0	Planificación	Definir alcance y cronograma	10 h	Alumno	100 %
Sprint 0	Planificación	Reunión con el tutor/cliente	5 h	Alumno	50 %
Sprint 0	Planificación	Ajuste de los entregables	6 h	Alumno	25 %
Sprint 1	HU1	Tarea 1 HU1	6 h / 3 SP	Alumno	0 %
Sprint 1	HU1	Tarea 2 HU1	10 h / 5 SP	Alumno	0 %
Sprint 2	HU2	Tarea 1 HU2	7 h / 5 SP	Alumno	0 %
...	...	...	...	...	...
Sprint 5	Escritura	Redacción memoria	50 h / 34 SP	Alumno	0 %
Sprint 6	Defensa	Preparación de la exposición	20 h / 13 SP	Alumno	0 %

11. Diagrama de Gantt (sprints)

Visualizar en un diagrama de Gantt la planificación temporal del proyecto, tomando como base los sprints definidos en la sección anterior. Debe contemplar todas las horas del proyecto.

Consigna:

- Elaborar un diagrama de Gantt que muestre la secuencia temporal de los sprints.
- Cada fila debe representar un sprint (con su número o nombre), y el eje horizontal debe indicar el tiempo (en semanas o fechas concretas).
- Las tareas técnicas derivadas de HU deben diferenciarse visualmente (por ejemplo, con un color distinto) de las tareas no técnicas (planificación, redacción, defensa).
- Incluir todas las tareas estimadas en cada sprint.

Recomendaciones para el Gantt:

- Podés usar herramientas gratuitas como TeamGantt, ClickUp, GanttProject, [Google Sheets], [Trello + Planyway], entre otras.
- Ordená los sprints de forma cronológica, comenzando con Sprint 0 (planificación) y finalizando con el sprint de defensa.
- Asegurate de reflejar la duración realista de cada sprint según tu disponibilidad y el cronograma general del posgrado.
- Incluí hitos importantes: reuniones, entregas parciales, defensa.

Incluir una imagen legible del diagrama de Gantt. Si es muy ancho, presentar primero la tabla y luego el gráfico de barras.

12. Gobernanza de datos

En esta sección se debe analizar de manera integral el marco normativo y ético asociado al uso de los datos y al desarrollo de soluciones basadas en inteligencia artificial.

El análisis se divide en dos partes:

- Cumplimiento normativo.
- Ética en el uso de inteligencia artificial.

12.1. Cumplimiento normativo

Este análisis es clave para garantizar el cumplimiento normativo y evitar conflictos legales durante el desarrollo y publicación del proyecto.

En esta subsección se debe identificar si los datos utilizados en el proyecto están sujetos a normativas de protección de datos y privacidad, y en qué condiciones pueden emplearse.

Aspectos a considerar:

- Determinar si los datos están regulados por normativas como el GDPR, la Ley 25.326 de Protección de Datos Personales (Argentina), la HIPAA u otras, según la jurisdicción o la temática del proyecto.
- Aclarar si las fuentes de datos son propias, de acceso público o licenciadas. En este último caso, explicitar la licencia correspondiente y sus condiciones de uso. Realizar este mismo análisis para las herramientas y, en caso de corresponder, para los modelos preentrenados a emplear.
- Analizar la viabilidad legal del proyecto, considerando los mecanismos necesarios para garantizar el cumplimiento normativo y la trazabilidad de los datos.

12.2. Ética en el uso de inteligencia artificial

En esta subsección se debe reflexionar sobre los aspectos éticos vinculados al uso de datos y algoritmos de inteligencia artificial. Se espera una evaluación de la integridad del sistema y su impacto social.

Aspectos a considerar:

- Identificar posibles sesgos en los datos o modelos (ideológicos, culturales, de género, geográficos, etc.) y analizar sus consecuencias (por ejemplo: discriminación, manipulación, pérdida de privacidad o desinformación).
- Analizar los posibles riesgos en términos de impacto social del uso indebido de la solución a desarrollar.
- En caso de corresponder, proponer medidas de mitigación y mecanismos de confianza (auditorías, documentación, trazabilidad, revisión humana, etc.).

- Finalmente, elaborar una reflexión general sobre la ética del proyecto, considerando tanto la equidad y transparencia del sistema como su impacto potencial en la sociedad.

13. Gestión de riesgos

a) Identificación de los riesgos (al menos cinco) y estimación de sus consecuencias:

Riesgo 1: detallar el riesgo (riesgo es algo que si ocurre altera los planes previstos de forma negativa)

- Severidad (S): mientras más severo, más alto es el número (usar números del 1 al 10). Justificar el motivo por el cual se asigna determinado número de severidad (S).
- Probabilidad de ocurrencia (O): mientras más probable, más alto es el número (usar del 1 al 10). Justificar el motivo por el cual se asigna determinado número de (O).

Riesgo 2:

- Severidad (S): X.
Justificación...
- Ocurrencia (O): Y.
Justificación...

Riesgo 3:

- Severidad (S): X.
Justificación...
- Ocurrencia (O): Y.
Justificación...

b) Tabla de gestión de riesgos: (El RPN se calcula como $RPN=S \times O$)

Riesgo	S	O	RPN	S*	O*	RPN*

Criterio adoptado:

Se tomarán medidas de mitigación en los riesgos cuyos números de RPN sean mayores a...

Nota: los valores marcados con (*) en la tabla corresponden luego de haber aplicado la mitigación.

c) Plan de mitigación de los riesgos que originalmente excedían el RPN máximo establecido:

Riesgo 1: plan de mitigación (si por el RPN fuera necesario elaborar un plan de mitigación).
Nueva asignación de S y O, con su respectiva justificación:

- Severidad (S*): mientras más severo, más alto es el número (usar números del 1 al 10). Justificar el motivo por el cual se asigna determinado número de severidad (S).
- Probabilidad de ocurrencia (O*): mientras más probable, más alto es el número (usar del 1 al 10). Justificar el motivo por el cual se asigna determinado número de (O).

Riesgo 2: plan de mitigación (si por el RPN fuera necesario elaborar un plan de mitigación).

Riesgo 3: plan de mitigación (si por el RPN fuera necesario elaborar un plan de mitigación).

14. Sprint Review

La revisión de sprint (*Sprint Review*) es una práctica fundamental en metodologías ágiles. Consiste en revisar y evaluar lo que se ha completado al finalizar un sprint. En esta instancia, se presentan los avances y se verifica si las funcionalidades cumplen con los criterios de aceptación establecidos. También se identifican entregables parciales y se consideran ajustes si es necesario.

Aunque el proyecto aún se encuentre en etapa de planificación, esta sección permite proyectar cómo se evaluarán las funcionalidades más importantes del backlog. Esta mirada anticipada favorece la planificación enfocada en valor y permite reflexionar sobre posibles obstáculos.

Objetivo: anticipar cómo se evaluará el avance del proyecto a medida que se desarrollen las funcionalidades, utilizando como base al menos cuatro historias de usuario del *Product Backlog*.

Seleccionar al menos 4 HU del Product Backlog. Para cada una, completar la siguiente tabla de revisión proyectada:

Formato sugerido:

HU seleccionada	Tareas asociadas	Entregable esperado	¿Cómo sabrás que está cumplida?	Observaciones o riesgos
HU1	Tarea 1	Módulo funcional	Cumple criterios de aceptación definidos	Falta validar con el tutor
	Tarea 2			
HU3	Tarea 1	Reporte generado	Exportación disponible y clara	Requiere datos reales
	Tarea 2			
HU5	Tarea 1	Panel de gestión	Roles diferenciados operativos	Riesgo en integración
	Tarea 2			
HU7	Tarea 1	Informe trimestral	PDF con gráficos y evolución	Puede faltar tiempo para ajustes
	Tarea 2			

15. Sprint Retrospective

La retrospectiva de sprint es una práctica orientada a la mejora continua. Al finalizar un sprint, el equipo (o el alumno, si trabaja de forma individual) reflexiona sobre lo que funcionó bien, lo que puede mejorarse y qué acciones concretas pueden implementarse para trabajar mejor en el futuro.

Durante la cursada se propuso el uso de la **Estrella de la Retrospectiva**, que organiza la reflexión en torno a cinco ejes:

- ¿Qué hacer más?
- ¿Qué hacer menos?
- ¿Qué mantener?
- ¿Qué empezar a hacer?
- ¿Qué dejar de hacer?

Aun en una etapa temprana, esta herramienta permite que el alumno planifique su forma de trabajar, identifique anticipadamente posibles dificultades y diseñe estrategias de organización personal.

Objetivo: reflexionar sobre las condiciones iniciales del proyecto, identificando fortalezas, posibles dificultades y estrategias de mejora, incluso antes del inicio del desarrollo.

Completar la siguiente tabla tomando como referencia los cinco ejes de la Estrella de la Retrospectiva (*Starfish* o estrella de mar). Esta instancia te ayudará a definir buenas prácticas desde el inicio y prepararte para enfrentar el trabajo de forma organizada y flexible. Se deberá completar la tabla al menos para 3 sprints técnicos y 1 no técnico.

Formato sugerido:

Sprint tipo y N°	¿Qué hacer más?	¿Qué hacer menos?	¿Qué mantener?	¿Qué empezar a hacer?	¿Qué dejar de hacer?
Sprint técnico - 1	Validaciones continuas con el alumno	Cambios sin versión registrada	Pruebas con datos simulados	Documentar cambios propuestos	Ajustes sin análisis de impacto
Sprint técnico - 2	Verificar configuraciones en múltiples escenarios	Modificar parámetros sin guardar historial	Perfiles reutilizables	Usar logs para configuración	Repetir pruebas manuales innecesarias
Sprint técnico - 8	Comparar correlaciones con casos previos	Cambiar parámetros sin justificar	Revisión cruzada de métricas	Anotar configuraciones usadas	Trabajar sin respaldo de datos
Sprint no técnico - 12 (por ej.: “Defensa”)	Ensayos orales con feedback	Cambiar contenidos en la memoria	Material visual claro	Dividir la presentación por bloques	Agregar gráficos difíciles de explicar